

Pan-European University, Faculty of Informatics

Interim Research Report

Definition of object-centric process

of the project

Requirements and formal definition of a low-code language based on object-centric processes - LowcodeOCP

Authors:

Gabriel Juhás, Milan Mladoniczky, Juraj Mažári and Tomáš Kováčik

Contact:

gabriel.juhas@paneurouni.com, milan.mladoniczky@paneurouni.com

Funded by the EU NextGenerationEU through the Recovery and Resilience Plan for Slovakia
under the project No.

09I03-03-V04-00493

June 30, 2026

Version: v0.2

Abstract

This report presents the first rigorous formal semantics for the Petriflow language, an executable modeling language designed for Process-Centric Objects (PCO). Unlike monolithic high-level Petri net formalisms, Petriflow utilizes strict encapsulation, where each object integrates its own process logic, data, and user interfaces. We formally define the task lifecycle through a token reservation mechanism and introduce a robust event-driven architecture. In this framework, events trigger actions that can, in turn, trigger further events, enabling the construction of complex transactions. This mechanism allows for the simulation of quasi-parallelism through event cascading while simultaneously supporting strictly sequential transactions, all while ensuring full transactional integrity. The presented formal framework provides a scalable foundation that effectively decouples declarative process design from its recursive, event-driven execution dynamics.

Contents

Contents	i
1 Introduction	1
2 Mathematical preliminaries	6
3 Petriflow process definition	7
4 Illustrative Example: Request Processing	21
4.1 Dynamic Interface Composition	21
4.2 Recursive Execution and Event Chaining	22
5 Conclusion	23
A Supplementary Material	27

Chapter 1

Introduction

In this section, we discuss requirements on a language for process-centric objects, resulting in definition of language called Petriflow, namely we discuss a definition of a process and inter-process communication. Briefly, Petriflow is based on extended Petri nets (where transitions represents tasks), enriched by data variables and data forms (associated with tasks).

This approach fundamentally differs from other high-level Petri net formalisms, such as Colored Petri Nets (CPN) [JK09], Object-Centric Petri Nets [AB20], or Reference Nets (used in Renew) [KWM04]. While these formalisms are powerful for modeling large-scale runtime systems, where cases or object instances are represented as colored tokens moving through a single, often monolithic net, they frequently lead to what we call a "spaghetti" model. In such models, a single net represents a "jungle" where all instances (animals) coexist, or a "forest" where all trees are modeled within one scope. This lack of encapsulation makes it difficult to distinguish between the blueprint and the instance, hindering compositionality and making it nearly impossible to replace one class definition with another without affecting the entire monolithic runtime structure.

In contrast, Petriflow shifts the focus from a runtime "map of the world" to a descriptive "catalogue of blueprints." Inspired by the Java class system, we define a **Petriflow class** not as the jungle itself, but as the DNA of a single species. Much like how a LEGO set provides the type definition for individual bricks that can later be composed into a house, a Petriflow class defines the discrete behavior, data, and lifecycle of a specific process-centric object. By strictly separating the *blueprint* (the Petriflow class) from its *realization* (the case or instance), we ensure high encapsulation and modularity. This allows developers to swap, extend, or interact with different classes through well-defined interfaces (events and actions) rather than managing a monolithic runtime model. This distinction is crucial: while traditional high-level nets excel at process mining and runtime visualization of existing systems, Petriflow is designed for the architectural modeling of new, complex, and highly maintainable software systems.

Despite its theoretical novelty, Petriflow is a language that has been used in the

industry for nearly a decade. Large number of process-centric objects currently run in large-scale applications across various sectors, including insurance claims processing, leasing automation, healthcare, utilities (energy and telecommunications), and the public sector. However, while Petriflow has achieved significant industrial success through the Netgrif platform, a rigorous formal definition has remained absent until now. This paper represents the first crucial step toward bridging this gap.

In this work, we focus on the formal definition of a single process-centric object and its internal behavior. We introduce the chaining of actions and the concept of tasks (and their interfaces) as sub-tasks within a single object. This modular approach is designed to be highly extensible; shifting toward multi-object systems, including one-to-many and many-to-many relationships, as well as cross-object event synchronization, will be a straightforward extension in our future work. Our formalization closely follows the actual Netgrif implementation to ensure that the model remains faithful to the practical success of the language. While we do not provide a full motivation for every design choice here, many elements are the result of years of empirical optimization. For instance, our sequential recursive chaining of events (using *pre* and *post* actions) is used to simulate the synchronous execution of event sets. In practice, this approach is more efficient than building complex multi-step parallel event sets, as it simplifies implementation while providing the flexibility to run transactions sequentially when full parallelism is not required.

Petriflow is an executable high-level modeling language for process-driven application development [Juh+21]. Petriflow follows the programming paradigm called process-driven programming (PDP) [Juh+19]. A comparison of Petriflow's concept with other well-known programming paradigms is essential to understanding the meaning of the PDP paradigm. Petriflow language combines the advantages of object-oriented programming (OOP), business process modeling (BPM) [ADO00; DPW04], event-driven programming (EDP), and relational databases (RDB).

Object-oriented programming languages brought about a distinguishing feature, encapsulation. This was a key step for the business world that needed new languages to solve the problem of growing application complexity. OOP was a concept that solved the problems that developers had with procedural and imperative programming. The concept of classes that contain data and methods strongly supports the modularity of applications.

While binding methods with the data in classes was one of the main features of OOP that helped to create more modular programs, Petriflow is binding a workflow process to a class to describe the life cycle of objects of the class.

The main building blocks of object-oriented programs are classes and their objects. In comparison, the main building blocks of process-driven programs in the Petriflow language are processes and their instances, called cases. A class is a blueprint of an object, and a Petriflow class is a blueprint of a Petriflow process-centric object, also called instance or

case. Simply, a Petriflow class is as a usual class with attributes enriched by a workflow process that defines the life cycle of the objects in that class. More accurately, a Petriflow class consists of data variables, their life cycle in form of a workflow process consisting of a flow of tasks and events and their actions. In the same way as in classes in OOP, data variables in a Petriflow class represent all attributes of its process-centric object. Data variables are typed. The change of the value of a data variable can be triggered by a so-called set event. Reading a value of a data variable can be triggered by a so-called get event.

Tasks are the active parts of objects of Petriflow classes. Data variables can be associated with workflow tasks to define data fields and create task forms. A data field, which is an association of a data variable to a task, is given as a rich relation that states:

1. whether a get event and/or a set event can trigger the data variable, i.e. whether its value is readable and/or editable,
2. whether the value of data variable is required,

Tasks have a simple life cycle: a task can be enabled, disabled or assigned to a user:

1. if a task is enabled, its change to the state assigned to a user can be triggered by a so-called assign-event
2. if a task is enabled, its readable data fields are accessible for reading by get-events
3. if a task is assigned to a user, its readable data fields are accessible for reading by get-events
4. if a task is assigned to a user, its editable data fields can be changed to values of the variable type by set-events
5. if a task is assigned to a user, its change to the state enabled or disable can be triggered by a so-called cancel-event
6. if a task is assigned to a user and all its required data fields have values of the variable type, its change to the state enabled or disabled can be triggered by a so-called finish-event.

Using the principles of event-driven programming, each data variable and each task is associated with an event listener: whenever an event triggers a change of the value of a data variable or whenever an event triggers a change of the task state, then a reaction can be defined by pieces of code (called actions) in the event listener. Whenever an event occurs, the actions in its event listener are executed. In actions, as a part of the code, events for other tasks and events for their data variables can be triggered. In this way, events and their reactions can create chains. Roles [BDM11] (lists of users) can be associated with tasks and their events, defining for each task which users are authorized to trigger events on that task. Some types of events are parametrized, for example a user triggering set data event of a variable must enter its value as a parameter. Another type

of a parametrized event is assign event of a task which has as a parameter a user to whom the task is assigned by the assign event. It means that whenever user A (within a role associated with assign event of a task t) triggers the assign event of t , he must choose a user B (different user or possibly himself) within role associated with the task t , to whom the task will be assigned. The assigned user B is then authorized to perform the task, i.e. to trigger set data events of editable data fields of the task t .

As a workflow process, Petriflow language uses place/transition Petri nets [DJ01; Des05; DR15] enriched by reset arcs [Aal+09], inhibitor arcs [AJ15] and read arcs [Vog02] to define the life cycle of the Petriflow process. Furthermore, Petriflow supports variable arc weights determined by data attribute values or place markings, a concept inspired by self-modifying nets [Val78]. Because reset arcs can be expressed by arcs with variable arc weights we will omit reset arcs in the formal definition. Places of the Petri net represent the control variables. As mentioned above, transitions of Petri net represent tasks of the life cycle of the workflow process. A task is enabled, whenever the corresponding transition in the underlying Petri net is enabled. The assign event with user B as a parameter, triggered on an enabled task by user A , consumes tokens from input places of the corresponding transition and assigns the task to user B . The finish event of the assigned task produces tokens to the output places of the corresponding transition. Triggering of cancel event of the assigned task returns consumed tokens back to input places of the corresponding transition. In this way, the workflow process defines when a task is enabled, assigned to a user or disabled. The life cycle of a Petriflow process is given by flow of assign/get/set/finish events on tasks and data variables respecting the restrictions on events given by the underlying Petri net and permissions in roles (lists of users).

When compared with relational databases, Petriflow classes correspond to tables, while objects (called also instances or cases) of the Petriflow class correspond to single records (rows) of a table. In a similar way to foreign keys in RDB and in a similar way to attributes of objects containing references to other objects in OOP, data variables of Petriflow classes can store the references to objects of Petriflow classes and references to the task and list of tasks of objects of Petriflow classes. In this way, one can easily share forms associated with one task as sub-forms within other tasks and implement a single source of truth architecture. Because in Petriflow references to lists of tasks can be stored also for tasks of the object itself. For sake of simplicity in this paper we only give a formal definition of one Petriflow process-centric object. In other words in the formal definition we do not distinguish between classes and their objects.

Process models in Petriflow language employ principles of object-oriented programming, relational databases and event-driven programming combining them with user authorization and the concept of the life cycle of objects using workflow processes borrowed from business process management. All this should bring a higher level of programming, with information about the data layer, application (process) layer and presentation layer (forms)

covered by a single Petriflow object with the aim to make the development of complex applications more structured, closer to the business user, faster, without the necessity to deal with the implementation details of the middleware. Netgrif Application Builder (NAB) is then a platform for developing process-driven applications. NAB produces process models in Petriflow language – a combination of XML and Groovy. This code can be directly interpreted in Netgrif Application Engine – Petriflow interpreter written in Java using the framework Spring Boot and storing the data of Petriflow process instances in MongoDB. Similarly to SQL build over relational databases, Petriflow language also provides a powerful query language that enables to create filters over process instances and their tasks [Juh+22; JJP23; Juh+26].

Chapter 2

Mathematical preliminaries

We use $\mathbb{N}$ to denote the nonnegative integers and $\mathbb{N}^+$ to denote the positive integers. Given two arbitrary sets A and B , the symbol B^A denotes the set of all functions from A to B . Given a function f from A to B and a subset C of A we write $f|_C$ to denote the restriction of f to the set C . The symbol 2^A denotes the power set of a set A . Given a binary relation $R \subseteq A \times A$ over a set A , the symbol R^+ denotes the transitive closure of R and R^* the reflexive and transitive closure of R . As is usual, for any set A , the symbol A^* denotes the set of all finite sequences of elements from A including the empty sequence.

Chapter 3

Petriflow process definition

Let us now define formal semantics for Petriflow processes.

Definition 1 establishes a *Petriflow process-centric object* (PCO) as a unified entity where data, logic, and user interaction are encapsulated into a single blueprint:

1. **Data (Information Layer):** Defines the object's state through a finite set of *Variables*. *Types* define the allowed domains for data. It introduces a unique *null* element to formally handle uninitialized states, while the *Typing* function ensures structural integrity by binding each variable to a specific domain.
2. **Workflow (Behavioral Layer):** Extends standard Petri nets to model complex life cycles. *Context Relations* representing read arcs (C_+) and inhibitor arcs (C_-), allow the model to verify the presence or absence of tokens without consumption. This mechanism is essential for implementing non-destructive "guards" and complex business rules. *Weight Function* enhances the net's expressive power by allowing arc weights to be dynamic. Weights can be constant values or dynamically evaluated based on place markings or data variable values, enabling self-modifying behavior driven by the current state.
3. **Interface (Interaction Layer):** Bridges internal logic with the end-user. It maps a subset of variables to each task, effectively defining a *Form*.

Definition 1 (Petriflow object)

A *Petriflow process-centric object* is a triple $PCO = (Data, Workflow, Interface)$, where:

1. *Data* is a triple $Data = (Variables, Types, Typing)$, such that:
 - (a) *Variables* is a finite set of data variables
 - (b) *Types* is a set of data types such that for each data type $V \in Types$, V is a set, called values of the data type V . We also consider a unique element *null* such that for each $V \in Types$ element $null \notin V$.

- (c) *Typing is a function $\text{Typing} : \text{Variables} \rightarrow \text{Types}$ associating a data type to each data variable.*
2. *Workflow is a six-tuple $\text{Workflow} = (P, T, F, C_+, C_-, W)$, where:*
- (a) *P is a finite set of places,*
 - (b) *T is a finite set of tasks, satisfying $P \cap T = \emptyset$,*
 - (c) *$F \subseteq (P \times T) \cup (T \times P)$ is a flow relation,*
 - (d) *$C_+, C_- \subseteq P \times T$ are positive and negative context relations,*
 - (e) *$W : F \cup C_+ \cup C_- \rightarrow \mathbb{N}^+ \cup P \cup \text{Variables}$ is a weight function.*
3. *Interface is a function that attaches to each $t \in T$ a subset of Variables, i.e. for each $t \in T : \text{Interface}(t) \subseteq \text{Variables}$. Given a task $t \in T$ elements of $\text{Interface}(t)$ are called data fields or data refs of the task t and $\text{Interface}(t)$ itself is also called a form of task t .*

Definition 2 establishes users, roles and tasklists:

1. **Users and Roles:** It formally integrates identity management into the PCO by defining roles as a non-empty subset of variables. Each role stores a set of users, allowing the model to dynamically manage permissions and authorizations within the data layer.
2. **Tasklists:** This introduces a specialized data type representing sets of tasks, enabling the PCO to handle collections of work items as first-class data objects. This is crucial for modeling task grouping, dynamic assignments, and sub-form structures.

Definition 2 (Users, Roles and Tasklists) *Given a PCO defined in Definition 1, let Users, Roles and Tasklists are defined as follows:*

1. *Let Users be a set such that $2^{\text{Users}} \in \text{Types}$ (2^{Users} are sets of users) and let Roles $\subseteq \text{Variables}$ such that $\text{Roles} = \{v \in \text{Variables} \mid \text{Typing}(v) = 2^{\text{Users}}\}$ is a nonempty set (i.e. values of Roles are sets of users, there is at least one Role).*
2. *Tasklists $= 2^T \in \text{Types}$, i.e. there is a data type with values being subsets of tasks.*

Definition 3 formalizes the runtime state s of a process-centric object as a total function covering control, data, and task-specific configurations:

1. **Control State ($s|_P$):** This represents the classic Petri net marking, mapping each place to a number of tokens. It tracks the current progression of the object through its defined lifecycle.

2. **Data State ($s(v)$):** This extends the state beyond the net structure by assigning a concrete value to each variable. By allowing the *null* value, the model formally handles incomplete information, which is critical for business process execution.
3. **Task State ($s(t)$):** This is a multi-dimensional state for each task:
 - (a) **Assignment ($s_{assigned}$):** Tracks which user is currently responsible for the task, enabling precise authorization and accountability.
 - (b) **Dynamic Permissions ($s_{editable}$):** This function determines the behavior of the form fields at runtime. It allows the PCO to dynamically change which data is modifiable based on the current context or previous actions.
 - (c) **Dynamic Mandatory Datafields ($s_{required}$):** This function determines which data is mandatory based on the current context or previous actions.
 - (d) **Consumption Tracking ($s_{consumed}$):** This sub-state tracks tokens that have been "locked" or consumed from input places when a task is assigned but not yet finished. It ensures the cancellation.

Definition 3 (State of Petriflow PCO) *Given a Petriflow PCO with users and roles and tasklists as defined in Definitions 1 and 2, a state of a Petriflow PCO is a function s with definition domain $P \cup Variables \cup T$ such that:*

1. $s|_P : P \rightarrow \mathbb{N}$ is a function that associates a nonnegative marking with each place,
2. for each $v \in Variables$: $s(v) \in Typing(v) \cup \{null\}$, i.e. state of each data variable is a value of the type of the variable or null,
3. for each $t \in T$: $s(t) = (s_{assigned}(t), s_{editable}(t), s_{required}(t), s_{consumed}(t))$ where
 - (a) $s_{assigned}(t) \in Users \cup \{null\}$ determines whether a task is assigned to a user, and
 - (b) $s_{editable}(t)$ is a function $s_{editable}(t) : Interface(t) \rightarrow \{0, 1\}$ determining for each data field of the task whether it is editable (if it is equal to 1) or not (if it is 0)
 - (c) $s_{required}(t)$ is a function $s_{required}(t) : Interface(t) \rightarrow \{0, 1\}$ determining for each data field of the task whether it is required (must have a value) (if it is 1) or can be unfilled (with value null) (if it is 0), and
 - (d) $s_{consumed}(t)$ is a function $s_{consumed}(t) : \{p \in P : (p, t) \in F\} \rightarrow \mathbb{N}$

Definition 4 describes the mechanism of *sub-forms* (embedded interfaces). It explains how a complex task form is dynamically composed from other tasks' interfaces through references:

- **Composition via Sub-forms:** A task can include other tasks (via *Tasklists*) as part of its own interface. We distinguish between "pure" data fields (actual values) and "references" (pointers to other tasks). The final form is a recursive collection of all pure fields found along these reference paths.
- **Propagation of Read-Only Status:** The logic follows a strict "inheritance of restrictions." If a parent task includes a sub-form as read-only, that entire sub-form (and all its own nested sub-forms) automatically becomes read-only. Once a path is "locked" at any level (*readonly**), the user cannot edit any data within that branch.
- **Mandatory Field Consolidation:** To ensure data integrity, if a data field is marked as "required" ($s_{required}$) in at least one part of the composed form, it is treated as required for the entire task ($s_{required}^*$). This prevents finishing a complex process if a crucial piece of data is missing anywhere in the hierarchy.
- **Termination and Safety:** Since the set of tasks is finite, the reflexive and transitive closure of references (*hasRef**) ensures that the form composition always terminates. There is no risk of infinite loops; the system simply builds the final interface from a finite number of building blocks.

Definition 4 (Extended Interface) *Given a Petriflow PCO with users, roles and tasklists as defined in Definitions 1 and 2, we define extended interface $Interface^*(t, s)$ of a task t in state s as follows:*

1. $PureInterface(t) = \{v \mid v \in Variables \wedge v \in Interface(t) \wedge Typing(v) \neq Tasklists\}$
2. $RefInterface(t) = \{v \mid v \in Variables \wedge v \in Interface(t) \wedge Typing(v) = Tasklists\}$
3. $hasRef(s) \subseteq T \times T$ is a binary relation such that

$$t \text{ hasRef}(s) t' \iff \exists v \in RefInterface(t) \wedge t' \in s(v)$$

establishing whether task t has reference to task t' in some Tasklist of task t .

4. $readonly : hasRef(s) \rightarrow \{0, 1\}$ such that for each $t \text{ hasRef}(s) t'$ there holds:

$$readonly(t, t') = 1 \iff \exists v \in RefInterface(t) : t' \in s(v) \wedge s_{editable}(t)(v) = 0$$

5. $hasRef^*(s)$ denotes reflexive and transitive closure of $hasRef(s)$
6. $readonly^* : hasRef^*(s) \rightarrow \{0, 1\}$ such that for each $t \text{ hasRef}(s)^* t'$ there holds

$$readonly^*(t, t') = 1$$

$$\Longleftrightarrow$$

$\exists$ a path $(t = t_0 \text{ hasRef}(s) \dots t_n = t')(n \in \mathbb{N}^+)$ such that

$\exists i \in \{0, \dots, n-1\} : \text{readonly}(t_i, t_{i+1}) = 1$

7. $\text{RefTasks}^*(t, s) = \{t' \in T \wedge t \text{ hasRef}^*(s) t'\}$

8. $\text{Interface}^*(t, s) = \bigcup_{t' \in \text{RefTasks}^*(t, s)} \text{PureInterface}(t')$

9. Given $v \in \text{Interface}^*(t, s)$, then $s_{\text{required}}^*(t)(v)$ is defined as follows:

if there exists $t' \in \text{RefTasks}^*(t, s)$ such that

$v \in \text{PureInterface}(t') \wedge s_{\text{required}}(t')(v) = 1$

then $s_{\text{required}}^*(t)(v) = 1$

otherwise $s_{\text{required}}^*(t)(v) = 0$

10. Given $v \in \text{Interface}^*(t, s)$, then $s_{\text{editable}}^*(t)(v)$ is defined as follows:

if there exists $t' \in \text{RefTasks}^*(t, s)$ such that

$v \in \text{PureInterface}(t') \wedge (s_{\text{editable}}(t')(v) = 0 \vee \text{readonly}^*(t, t') = 1)$

then $s_{\text{editable}}^*(t)(v) = 0$

otherwise $s_{\text{editable}}^*(t)(v) = 1$

Definition 5 establishes events and mapping of roles on events:

- **Role Mapping (1.a,c,e,g,i):** Every operational event in a task is protected by a specific role. This mapping (*RoleRef*) ensures that for every action, whether it is viewing a form, assigning a task to a specific user, canceling, or finishing it, the system knows exactly which group of users is authorized to perform it.
- **Task Lifecycle Events (1.b,d,f,h):** The definition formalizes the discrete stages of a task. The *assign* event is special because it is "parametrized", the user performing the assignment must choose a target user who belongs to the required role, establishing clear accountability.
- **Data Events (2):** Access to information is also event-driven. Data variables are not just static values; they are interacted with through *set* events (to change a value with a specific parameter) and *get* events (to read a value). This allows the system to trigger specific actions or validations whenever data is accessed or modified.

Definition 5 (Role References and Events of a Petriflow PCO) *Given a Petriflow PCO with users and roles and tasklists as defined in Definitions 1 and 2. Let RoleRef be a mapping such that:*

1. for each $t \in T$:

- (a) $RoleRef(t) \in Roles$
- (b) $view_t$ denotes a view event
- (c) $RoleRef(view_t) \in Roles$
- (d) $assign_t(u)$ is a parametrized assign event with parameter $u \in RoleRef(t)$,
- (e) $RoleRef(assign_t) \in Roles$
- (f) $cancel_t$ denotes a cancel event
- (g) $RoleRef(cancel_t) \in Roles$
- (h) $finish_t$ denotes a finish event
- (i) $RoleRef(finish_t) \in Roles$

2. for each $v \in Variables$:

- (a) $set_v(x)$ denotes a parametrized set-data event with parameter $x \in Typing(v) \cup \{null\}$
- (b) get_v denotes a get-data event.

Definition 6 formalizes in what state a task is enabled. A task t is enabled in state s if and only if:

- **Assignment Status (1):** The task is not currently assigned to any user ($s_{assigned} = null$).
- **Constant Flow and Read Arcs (2):** As is standard in Petri nets with read arcs, for each regular or read arc with constant weight from place p to t , the number of tokens in p must be at least the arc weight.
- **Variable Weights (3):** For each regular or read arc with variable weight (via associated data variable or place) from place p to t , the number of tokens in p must be at least the current value of the associated data variable (which must be a non-negative integer) or the current marking of the associated place.
- **Constant Inhibitor Arcs (4):** Following standard inhibitor logic, for each inhibitor arc with constant weight from place p to t , the number of tokens in p must be strictly lower than the weight of the arc.
- **Variable Inhibitor Arcs (5):** For each inhibitor arc with variable weight (via associated data variable or place) from place p to t , the number of tokens in p must be lower than the current value of the associated data variable (which must be a non-negative integer) or the current marking of the associated place.

Definition 6 (Enabled task) *Given a Petriflow PCO as defined in Definition 1, we define that a task t is enabled in state s of the Petriflow PCO $\iff$:*

1. $s_{assigned}(t) = null$ and
2. for each $(p, t) \in F \cup C_+$ such that $W((p, t)) \in \mathbb{N}^+$ there holds $s(p) \geq W((p, t))$ and
3. for each $(p, t) \in F \cup C_+$ such that $W((p, t)) \in P \cup Variables$ there holds $s(W((p, t))) \in \mathbb{N} \wedge s(p) \geq s(W((p, t)))$ and
4. for each $(p, t) \in C_-$ such that $W((p, t)) \in \mathbb{N}^+$ there holds $s(p) < W((p, t))$ and
5. for each $(p, t) \in C_-$ such that $W((p, t)) \in P \cup Variables$ there holds $s(W((p, t))) \in \mathbb{N} \wedge s(p) < s(W((p, t)))$.

Definition 7 formalizes the conditions under which a user can view the interface of a task. It is an event that does not change the state of the PCO:

- **Authorization (1):** A user u can trigger a view event only if user u belongs to the role authorized for that specific task's view (*RoleRef*).
- **Visibility Conditions (1):** Unlike assignment, a task can be viewed in two cases: either when the task is *enabled* (available for everyone in the role) or when it is *already assigned* to someone ($s_{assigned} \neq null$). This allows managers or other authorized users to see the state of a task even while someone else is working on it.
- **State Invariance:** Triggering a view event is purely observational. It leaves the state of the PCO (marking, data, and assignments) completely unchanged: $trigger(s, view_t) = s$.

Definition 7 (Triggering a view event) *Given a Petriflow PCO with users, roles and tasklists as defined in Definitions 1 and 2, and RoleRefs in Definition 5, we define triggering of a view event by a user $u \in Users$ in state s of the Petriflow PCO as follows:*

1. for each $t \in T$ event $view_t$

can be triggered by user u in state s

$\iff$

$u \in RoleRef(view_t)$ and $(t \text{ is enabled in state } s \text{ or } s_{assigned}(t) \neq null)$

When a view event $view_t$ can be triggered by user u in state s then its triggering does not change the state of the PCO, i.e. we define $trigger(s, view_t) = s$.

Definition 8 formalizes the act of "locking" a task for execution. Unlike standard Petri nets where firing is instantaneous, Petriflow splits the process into phases to handle human interaction:

- **Who and What (1):** A user u (typically a system or manager) can assign a task t to a user x (who must belong to the role $RoleRef(t)$) only if u has the authority to perform the assignment (u belong to the role $RoleRef(assign_t)$) and the task is currently enabled.
- **Token Locking (2–3):** When a task is assigned, the required tokens are immediately removed from the input places. Instead of disappearing, they are stored in a special buffer, $s_{consumed}(t)$ for canceling. This applies to both constant weights and dynamic weights derived from variables or other places.
- **Exclusive Ownership (4):** The task's state is updated to $s'_{assigned}(t) = x$. This marks the task as being "in progress" by a specific person, effectively preventing any other user from claiming the same task until it is either finished or canceled.

Definition 8 (Triggering an assign event) *Given a Petriflow PCO with users, roles and tasklists as defined in Definitions 1 and 2, and RoleRefs in Definition 5, we define triggering of an assign event by a user $u \in Users$ in state s of the Petriflow PCO as follows:*

1. for each $t \in T$ event $assign_t(x)$ with user $x \in RoleRef(t)$ as a parameter

can be triggered by user u in state s

$\iff$

$u \in RoleRef(assign_t)$ and t is enabled in state s .

When an assign event $assign_t(x)$ can be triggered by user u in state s then its triggering leads to a new state $trigger(s, assign_t(x)) = s'$ that differs from state s as follows:

2. for each $(p, t) \in F$ such that $W((p, t)) \in \mathbb{N}$ there holds $s'(p) = s(p) - W((p, t))$ and $s'_{consumed}(t)(p) = W((p, t))$ and
3. for each $(p, t) \in F$ such that $W((p, t)) \in P \cup Variables$ there holds $s'(p) = s(p) - s(W((p, t)))$ and $s'_{consumed}(t)(p) = s(W((p, t)))$ and
4. $s'_{assigned}(t) = x$.

Definition 9 formalizes the mechanism for aborting a task.

- **Authorization (1):** A cancel event can be triggered only by an authorized user (based on $RoleRef(cancel_t)$) and only if the task is currently assigned to someone ($s_{assigned}(t) \neq null$).
- **Token Rollback (2):** All consumed tokens previously "locked" in the buffer ($s_{consumed}(t)$) are returned to their original input places. It is important to note that changes caused by data variables are not reverted by a cancel event in the current version of Petriflow. While a simple state rollback would be technically feasible, the true complexity arises from chained actions (pre- and post-events). These can trigger side effects, such as assignments of other tasks or external system calls, that are practically impossible to revert once executed. Managing such complex causal chains in real-world applications remains a significant challenge that nine years of research have yet to resolve satisfactorily.
- **Releasing the Task (3):** The task's assignment is cleared ($s'_{assigned}(t) = null$). This makes the task "publicly" enabled again, allowing any eligible user in the role to claim and start it anew.

Definition 9 (Triggering a cancel event) *Given a Petriflow PCO with users, roles and tasklists as defined in Definitions 1 and 2, and RoleRefs in Definition 5, we define triggering of an cancel event by a user $u \in Users$ in state s of the Petriflow PCO as follows:*

1. for each $t \in T$ event $cancel_t$

can be triggered by user u in state s

$\iff$

$u \in RoleRef(cancel_t) \text{ and } s_{assigned}(t) \neq null$

When a cancel event $cancel_t$ can be triggered by user u in state s then its triggering leads to a new state $trigger(s, cancel_t) = s'$ that differs from state s as follows:

2. for each $(p, t) \in F$ there holds $s'(p) = s(p) + s_{consumed}(t)(p)$ and
3. $s'_{assigned}(t) = null$.

Definition 10 defines the final stage of the task lifecycle:

- **Execution Guard (1):** A task can only be finished if the user u has the required role. Crucially, the system checks the *Extended Interface* denoted as $(Interface^*(t, s))$: any data field marked as required, even those originating from nested sub-forms, must have a non-null value. Additionally, if any output arc has a dynamic weight, its reference must resolve to a valid non-negative integer.

- **Token Production (2–3):** Upon finishing, the process state is updated by producing new tokens in the output places. Similar to the enablement rules, the number of tokens produced can be a fixed constant or a dynamic value pulled from data variables or marking of places.
- **Resource Release (4):** The task assignment is cleared ($s'_{assigned}(t) = null$), and since the "locked" tokens in $s_{consumed}(t)$ were effectively consumed by the transition, the task returns to an idle state.

Definition 10 (Triggering a finish event) *Given a Petriflow PCO with users, roles, tasklists, and extended interface as defined in Definitions 1, 2 and 4, and RoleRefs in Definition 5, we define triggering of an finish event by a user $u \in Users$ in state s of the Petriflow PCO as follows:*

1. for each $t \in T$ event $finish_t$ can be triggered by user u in state $s \iff$

(a) $u \in RoleRef(finish_t)$ and

(b) $s_{assigned}(t) \neq null$ and

(c) for each $v \in Interface^*(t, s) : s^*_{required}(t)(v) = 1 \implies s(v) \neq null$ and

(d) for each $(t, p) \in F$ such that $W((t, p)) \in P \cup Variables$ there holds $s(W((t, p))) \in \mathbb{N}$.

When a finish event $finish_t$ can be triggered by user u in state s then its triggering leads to a new state $trigger(s, finish_t) = s'$ that differs from state s as follows:

2. for each $(t, p) \in F$ such that $W((t, p)) \in \mathbb{N}$ there holds $s'(p) = s(p) + W((t, p))$ and

3. for each $(t, p) \in F$ such that $W((t, p)) \in P \cup Variables$ there holds $s'(p) = s(p) + s(W((t, p)))$ and

4. $s'_{assigned}(t) = null$

Definition 11 formalizes how data variables are updated within the PCO. It ensures that data integrity is maintained by strictly linking data modification to task ownership and permissions:

- **Strict Context for Editing (1):** A user u cannot simply change any variable at any time. A *set* event is only valid if:
 - The user is currently assigned to a task t that acts as the "editor".
 - The variable v is part of that task's extended interface ($Interface^*(t, s)$), meaning it is either directly in the task or in one of its sub-forms.

- The field is explicitly marked as editable ($s_{editable}^*(t)(v) = 1$) in the current state.
- The new value is different from the current one (preventing redundant updates).
- **Atomic Data Update (2):** When these conditions are met, the state of the variable is updated to the new value ($s'(v) = value$).

Definition 11 (Triggering a set event) *Given a Petriflow PCO with users, roles and tasklists as defined in Definitions 1, 2 and 4, we define triggering of a set event by a user $u \in Users$ in state s of the Petriflow PCO as follows:*

1. for each $v \in Variables$ and each $value \in Typing(v) \cup \{null\}$

event $set_v(value)$ can be triggered by user u in state s

$\iff$

there exists a task $t \in T$ such that

$s_{assigned}(t) = u$ and $v \in Interface^(t, s)$ and*

$s_{editable}^(t)(v) = 1$ and $value \neq s(v)$*

When $set_v(value)$ can be triggered by user u in state s then its triggering leads to a new state $trigger(s, set_v(value)) = s'$ that differs from state s as follows:

2. $s'(v) = value$.

Definition 12 defines the conditions for data retrieval. It ensures that variables are not globally exposed but are accessed only through the context of a task's interface:

- **Authorized Access (1):** A user u can trigger a *get* event for a variable v only if there exists a task t such that:
 - The user is authorized to view the task ($u \in RoleRef(view_t)$).
 - The task is in an active state, either currently enabled or already assigned to a user.
 - The variable v is explicitly part of extended interface ($Interface^*(t, s)$) of that task, potentially including nested data from sub-forms.
- **State Invariance:** Similar to a view event, a *get* event is purely informational. It does not alter the marking, the variable values, or the assignments within the PCO: $trigger(s, get_v) = s$.

Definition 12 (Triggering a get event) *Given a Petriflow PCO with users, roles and tasklists as defined in Definitions 1, 2 and 4, we define triggering of a get event by a user $u \in Users$ in state s of the Petriflow PCO as follows:*

1. for each $v \in Variables$

event get_v can be triggered by user u in state s

$\iff$

there exists a task $t \in T$ such that :

$u \in RoleRef(view_t)$ and (t is enabled in s or $s_{assigned}(t) \neq null$) and

$v \in Interface^(t, s)$*

Similarly to view events, when a get event get_v can be triggered by user u in state s then its triggering does not differ from state s , i.e. we define $trigger(s, get_v) = s$.

Definition 13 introduces the operational "logic" of Petriflow, allowing events to trigger additional sequences of events. This mechanism enables automation and complex business rules:

- **Sequence of Events (1):** An *Action* is defined as a finite sequence of events (E^*). This means that one single trigger (like clicking a button) can result in a whole cascade of further operations, such as setting multiple variables or assigning new tasks.
- **Pre-actions and Post-actions (2):** Every event in the system is wrapped in two logic hooks:
 - **Pre-action (*pre*):** A function that determines which sequence of events should be executed *before* the main event itself is processed. This is typically used for validation or preparation of the state.
 - **Post-action (*post*):** A function that determines the sequence of events to be executed *after* the main event has successfully completed. This is where automation, side effects, and "chained" process logic usually reside.
- **State-Dependency:** Crucially, both *pre* and *post* are functions on states S . This means the system can decide to trigger different actions based on the current data values or the marking of the net.

Definition 13 (Actions) *Given a Petriflow PCO with events as defined in Definitions 1 and 5, by E we denote all events, by S we denote all states of PCO and we define Actions on events as follows:*

1. Action is a finite and possible an empty sequence of events, i.e. $action \in E^*$
2. Each event $e \in E$ has pre-action pre and post-action $post$ such that:
 - (a) $pre : S \times E \rightarrow E^*$ is a function that assigns an action called pre-action to each state and each event
 - (b) $post : S \times E \rightarrow E^*$ is a function that assigns an action called post-action to each state and each event

Definition 14 describes the recursive execution engine of Petriflow. It formalizes how a single user interaction can trigger a cascading chain of automated events, ensuring transactional integrity through a fail-fast mechanism:

- **Recursive Execution Logic:** The execution is governed by a set of mutually recursive functions. When an event e is triggered, the system first executes the entire sequence of *pre-actions*, then the *main event*, and finally the sequence of *post-actions*.
- **Deep Cascading and Termination:** Because *execute_sequence* calls *execute_event_with_actions* for each sub-event, Petriflow supports nested logic. It is important to note that in its current form, Petriflow does not check for termination. Ensuring that these chains do not result in infinite loops is the responsibility of the developer.
- **Transactional Stability ($\perp$):** The symbol $\perp$ represents an invalid state. If any event in the entire chain, whether a pre-action, the main event, or a deeply nested post-action, cannot be triggered, the entire execution fails and returns $\perp$.
- **Transactional Execution:** An event is only considered successfully triggered if the entire tree of associated actions is completed. This ensures the system remains consistent, preventing partial updates.

Definition 14 (Executing Event with Actions) *Given a Petriflow PCO with events as defined in Definitions 1 and 5. Let E be the set of events, S the set of states and $\perp \notin S$, which also means no event can be triggered in $\perp$. Using *execute* function and *execute_sequence* function as defined by pseudocode below, execution of an event with actions is given by the pseudocode of function *execute_event_with_actions*:*

```

execute_event_with_actions( $s \in S \cup \{\perp\}, e \in E$ ) {
  if  $e$  can be triggered in  $s$  {
     $s \leftarrow \text{execute\_sequence}(s, \text{pre}(e))$ 
     $s \leftarrow \text{execute}(s, e)$ 
     $s \leftarrow \text{execute\_sequence}(s, \text{post}(e))$ 
    return  $s$ 
  }
  return  $\perp$ 
}

execute( $s \in S \cup \{\perp\}, e \in E$ ) {
  if  $e$  can be triggered in  $s$ 
    return  $\text{trigger}(s, e)$ 
  else
    return  $\perp$ 
}

execute_sequence( $s \in S \cup \{\perp\}, \sigma \in E^*$ ) {
  for ( $e$  in  $\sigma$ ) { // iteratively do for each event in sequence  $\sigma$ 
     $s \leftarrow \text{execute\_event\_with\_actions}(s, e)$ 
  }
  return  $s$ 
}

```

Finally, for each state $s \in S$ and each event $e \in E$:

event e can be executed in state s

$\iff$

$\text{execute_event_with_actions}(s, e) \neq \perp$

Then execution of event e in state s leads to state

$s' = \text{execute_event_with_actions}(s, e)$.

Chapter 4

Illustrative Example: Request Processing

To demonstrate the synergy between the extended interface mechanism (Definition 4) and the recursive event-driven execution (Definition 14), we present a request processing scenario with lifecycle depicted in Figure 4.1.

4.1 Dynamic Interface Composition

Consider a task `request_form` that acts as a data container, including variables such as *name*, *surname*, *email*, *phone_number*, *text_request*, and *attachments*, that are editable in the task `request_form`. Petriflow handles the visibility and editability of these fields across different process stages:

- **Editable Propagation:** The task `submit_request` contains a data variable of type *Tasklists* called *reference_to_request_form* pointing to task `request_form`. Since this data variable is marked as **editable** in the task `submit_request`, Definition 4 ensures that all fields within the referenced sub-form remain editable. Furthermore, variables marked as **required** in the `request_form` (e.g., *email* and *text_request*) remain required in the task `submit_request`. The user cannot trigger the *finish* event until these fields are filled.
- **Read-Only Inheritance:** In the `register` task, the same data variable *reference_to_request_form* is used but marked as **visible** (read-only). Following the *readonly** relation in Definition 4, all sub-fields are automatically rendered as non-editable, ensuring the registrar can view the submitted data but cannot modify it.

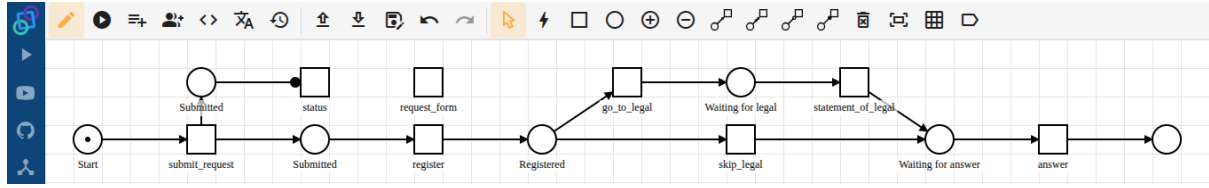


Figure 4.1: Lifecycle of request process-centric object demonstrating the synergy between the extended interface mechanism (Definition 4) and the recursive event-driven execution (Definition 14).

4.2 Recursive Execution and Event Chaining

The execution of the finish event of the `register` task illustrates the event chaining power described in Definition 14:

- **Pre-action Logic:** The *finish* event of the `register` task contains a pre-action that updates the `request.state` variable to the string *"Your request was registered"*. This variable is contained as non-editable in the `status` task and after execution of the `register` task this new value can be seen by a user triggering the *view* event of the task `status` and *get* event of the data variable `request.state`.
- **Conditional Chaining:** Given that $boolean = \{0, 1\} \in Types$, and data variable `to_legal` with Typing *boolean* is contained in the `register` task as editable and required:
 - the *finish* event's post-action of the `register` task for any state with $s(to_legal) = 1$ triggers a sequence of *assign* and *finish* events for the `go_to_legal` task.
 - the *finish* event's post-action of the `register` task for any state with $s(to_legal) = 0$ triggers a sequence of *assign* and *finish* events for the `skip_legal` task.
- **Transactional Integrity:** This entire cascade is executed via the function `execute_event_with_actions`. If any nested event fails, the entire chain is aborted, and the system returns $\perp$, preventing a partial or inconsistent process state.

Chapter 5

Conclusion

In this paper, we have presented the first rigorous formal semantics for the Petriflow language, bridging a decade-long gap between its extensive industrial application and its theoretical grounding. By defining the Petriflow Process-Centric Object (PCO), we have established a framework that integrates workflow logic, data variables, and user-driven interfaces into a single encapsulated blueprint.

Our formalization highlights several critical aspects of the language's execution model:

- **Encapsulation and Modularity:** Unlike monolithic high-level Petri net formalisms, Petriflow treats each process as a discrete class (DNA), allowing for high modularity and easier maintenance of complex software systems.
- **Transactional Execution:** Through the recursive execution of pre-actions, core events, and post-actions defined in Definition 14, Petriflow ensures that any operational chain either completes successfully or fails entirely, returning $\perp$ to maintain a consistent state.
- **Operational Realities:** We have formally addressed the complexity of causal chains. While the system provides a robust rollback for place markings via token locking (Definition 9), it intentionally does not revert changes to data variables due to the inherent difficulty of undoing side effects (such as external system calls) in real-world applications. As noted, managing these chains remains a challenge that nine years of research have yet to resolve satisfactorily.
- **Scalability of the Formalism:** A key strength of the presented model is that its core mechanisms, specifically the recursive execution of the function `execute_event_with_actions` and the construction of the extended interface (*Interface**), are designed to remain invariant under system expansion.

The definitions presented here serve as the necessary foundation for a single-object system. It is important to emphasize that while the current Netgrif implementation

already supports multi-object interactions and inter-process communication, as illustrated in our companion work at Petri Nets 2026 [Juh+26], our future work will focus on formally extending this definition.

This extension is envisioned as a natural scaling of the current model: the state space S and the set of events E will expand into a disjoint union of states and events across multiple objects. Crucially, the fundamental logic of task referencing (sub-forms from different objects) and cross-object event triggering will leverage the same recursive and interface-building principles established in this paper. This evolution will finalize the formal roadmap for Petriflow process-centric classes and process-centric objects as a comprehensive language for process-driven programming.

Bibliography

- [AB20] Wil M. P. van der Aalst and Alessandro Berti. “Discovering Object-Centric Petri Nets”. In: *CoRR* abs/2010.02047 (2020). DOI: 10.48550/arXiv.2010.02047. arXiv: 2010.02047.
- [ADO00] Wil M. P. van der Aalst, Jörg Desel, and Andreas Oberweis, eds. *Business Process Management, Models, Techniques, and Empirical Studies*. Vol. 1806. Lecture Notes in Computer Science. Springer, 2000. DOI: 10.1007/3-540-45594-9.
- [Aal+09] Wil M. P. van der Aalst et al. “Soundness of Workflow Nets with Reset Arcs”. In: *Transactions on Petri Nets and Other Models of Concurrency III*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 50–70. ISBN: 978-3-642-04856-2. DOI: 10.1007/978-3-642-04856-2_3.
- [AJ15] Mohammed Alqarni and Ryszard Janicki. “On Interval Process Semantics of Petri Nets with Inhibitor Arcs”. In: *Application and Theory of Petri Nets and Concurrency*. Springer International Publishing, 2015, pp. 77–97. ISBN: 978-3-319-19488-2.
- [BDM11] Robin Bergenthum, Jörg Desel, and Sebastian Mauser. “Workflow Nets with Roles”. In: *EMISA*. Vol. P-190. LNI. GI, 2011, pp. 65–78.
- [Des05] Jörg Desel. “Process Modeling using Petri Nets”. In: *Process-Aware Information Systems: Bridging People and Software through Process Technology* (Oct. 2005), pp. 147–177. DOI: 10.1002/0471741442.ch7.
- [DJ01] Jörg Desel and Gabriel Juhás. ““What Is a Petri Net?” Informal Answers for the Informed Reader”. In: *Unifying Petri Nets: Advances in Petri Nets*. Ed. by Hartmut Ehrig et al. Springer Berlin Heidelberg, 2001, pp. 1–25. ISBN: 978-3-540-45541-7. DOI: 10.1007/3-540-45541-8_1.
- [DPW04] Jörg Desel, Barbara Pernici, and Mathias Weske, eds. *Business Process Management: Second International Conference, BPM 2004, Potsdam, Germany, June 17-18, 2004. Proceedings*. Vol. 3080. Lecture Notes in Computer Science. Springer, 2004. DOI: 10.1007/b98280.

- [DR15] Jörg Desel and Wolfgang Reisig. “The concepts of Petri nets”. In: *Softw. Syst. Model.* 14.2 (2015), pp. 669–683. ISSN: 1619-1366. DOI: 10.1007/s10270-014-0423-3.
- [JK09] Kurt Jensen and Lars M. Kristensen. *Coloured Petri Nets: Modelling and Validation of Concurrent Systems*. Springer Science & Business Media, 2009. DOI: 10.1007/b95112.
- [Juh+21] G. Juhás et al. “Petriflow language and Netgrif Application Builder”. In: *BPM (PhD/Demos)*. Vol. 2973. CEUR Workshop Proceedings. CEUR-WS.org, 2021, pp. 171–175. URL: https://ceur-ws.org/Vol-2973/paper_284.pdf.
- [JJP23] Gabriel Juhás, Ana Juhásová, and Lubos Petrovic. “Low-Code Languages in IT Education: Integrating Theory and Practice”. In: *2023 21st International Conference on Emerging eLearning Technologies and Applications (ICETA)* (2023), pp. 249–257. DOI: 10.1109/ICETA61311.2023.10343807.
- [Juh+22] Gabriel Juhás et al. “Low-code platforms and languages: the future of software development”. In: *2022 20th International Conference on Emerging eLearning Technologies and Applications (ICETA)* (2022), pp. 286–293. DOI: 10.1109/ICETA57911.2022.9974697.
- [Juh+26] Gabriel Juhás et al. “Netgrif Platform: A Tool for Executable Models of Object-Centric Processes in Petriflow Language”. In: *Application and Theory of Petri Nets and Concurrency*. Springer Nature Switzerland, 2026, pp. 371–382. ISBN: 978-3-032-27879-1. DOI: 10.1007/978-3-032-27879-1_19.
- [Juh+19] Ana Juhasova et al. “IT Induced Innovations: Digital Transformation and Process Automation”. In: *Proc. ICETA 2019 - 17th IEEE International Conference on Emerging eLearning Technologies and Applications*. IEEE, Nov. 2019, pp. 322–329. DOI: 10.1109/ICETA48886.2019.9040004.
- [KWM04] Olaf Kummer, Frank Wienberg, and Daniel Moldt. “Renew: The Reference Net Workshop”. In: *Tool Presentations, 25th International Conference on Applications and Theory of Petri Nets*. 2004.
- [Val78] Rüdiger Valk. “Self-modifying nets, a natural extension of Petri nets”. In: *Automata, Languages and Programming*. Springer Berlin Heidelberg, 1978, pp. 464–476. ISBN: 978-3-540-35807-7. DOI: 10.1007/3-540-08860-1_35.
- [Vog02] Walter Vogler. “Partial order semantics and read arcs”. In: *Theoretical Computer Science* 286.1 (2002). Mathematical Foundations of Computer Science, pp. 33–63. ISSN: 0304-3975. DOI: [https://doi.org/10.1016/S0304-3975\(01\)00234-1](https://doi.org/10.1016/S0304-3975(01)00234-1).

Appendix A

Supplementary Material

For newest version of this report and other supplementary material including XML scheme of the syntax of the Petriflow OCP see petriflow.org.